

INTERNAL ENGINEERING SNAPSHOT

ORCHIDS · PROJECT GREEN

Engineering Session Snapshot

Two strict QA cycles, a real automated test suite built from scratch, GitHub Actions CI wired up end-to-end, and a security hygiene pass — a full account of what changed and what's still open.

DATE

03 July 2026

BRANCHES

develop → main

BUGS FIXED

24 total

COMPLETION

~90 / 100

Table of Contents

01	Executive Summary	3
02	QA Cycle 1 — 5 Bugs Found & Fixed	4
03	QA Cycle 2 — 10 Bugs Found & Fixed	5
04	Infrastructure & Environment Fixes	7
05	The Automated Test Suite	8
06	8 More Bugs the Suite Caught	9
07	CI/CD on GitHub Actions	11
08	Security & Repo Hygiene	12
09	Git Workflow Note	12
10	Completion Scorecard	13
11	What's Left To Do	14

Naming convention used for this document: PROJECT-GREEN-SESSION-SNAPSHOT-2026-07-03.pdf , following the existing pattern in docs/ (e.g. QA_FULL_SYSTEM_TEST_REPORT_2026-07-04.md , QA_FIX_VERIFICATION_2026-07-03.md , SYSTEM-SNAPSHOT-2026-07-02-2200.md) — <SCOPE>-SNAPSHOT-<YYYY-MM-DD> , uppercase, hyphen-separated, dated.

Executive Summary

This session ran a strict, full-system QA pass on Project Green twice, then went further than QA alone: it built a real automated test suite from nothing, wired up continuous integration on GitHub, and closed a secrets-hygiene gap — each stage catching genuine bugs the previous stage hadn't.

QA Cycle 1 — 5 bugs found and fixed the same day, centred on the buyer order-detail page and downstream notification/invoice effects.

QA Cycle 2 — a fresh, independent strict pass found 10 more bugs, including the Delivery Coordinator role having no web portal at all (P0).

Automated test suite — every `*.test.js` file was previously an empty stub. Built 57 real tests across 20 modules; writing them surfaced **8 more genuine bugs**.

CI on GitHub Actions — first real run caught **one more bug** no local check had: a tsconfig/jsconfig mismatch that only failed on a clean checkout.



Running total

Stage	Bugs found & fixed	Verification method
QA Cycle 1 (07-02/07-03)	5	Live API calls + direct PostgreSQL queries
QA Cycle 2 (07-03 afternoon)	10	Live API calls + direct PostgreSQL queries + browser session
Writing the automated test suite	8	Real node:test integration tests, re-run to green
First GitHub Actions CI run	1	Clean-checkout build failure, reproduced & fixed locally
Total	24	—

Overall completion estimate: ~90 / 100. Core commerce flow, all 6 role portals, backend test coverage, and CI are solid. The clearest remaining gaps are real product images (currently 0 of 523 working), a manual frontend QA pass, and automated frontend test coverage — see §11.

5 Bugs Found & Fixed

The first strict full-system QA pass centred on the buyer's own order-detail page and the money/notification effects downstream of it.

P0 Buyer order-detail page — doubly (actually quadruply) broken

Response unwrapping used `res.data` instead of `res.data.data`, silently nulling every field. Cancel/Confirm-Receipt buttons called `api.post(...)` against routes registered as `PATCH`. Status comparisons used fictional enum values (`PENDING`, `CONFIRMED`, `SHIPPED`) instead of the real ones. Fields were read in camelCase (`orderNo`, `unitPrice`) when the API only ever returns snake_case (`order_no`, `unit_price_at_order`). All four fixed together; the linked-invoice card, which never had a data source at all, was wired up too.

P2 POD upload faked buyer confirmation

`delivery.repository.js` set `buyer_confirmed_at` the instant a courier uploaded a proof-of-delivery photo — no buyer involved. Moved that write to the real, buyer-only `confirmReceipt()` action.

P1 Cancelling an approved order left its invoice as a live receivable

Cancelling an `APPROVED` order now voids (or adjusts, if partially paid) its invoice in the same transaction, instead of permanently inflating the buyer's credit exposure.

P2 Upload validation leaked a raw 500 + stack trace

A bad file type on any upload endpoint (products, CMS media, delivery POD) now returns a clean `400 INVALID_FILE_TYPE` instead of an unhandled exception.

P2 Notifications had no recipient

`payment_received` referenced a non-existent `invoice.recipient_email` field; `price_approval_needed` hardcoded a null recipient. Both now resolve real emails (buyer via `trade_accounts → users`; every active admin, one outbox row each) and were confirmed reaching `SENT` status.

10 Bugs Found & Fixed

A second, independent strict pass — this one caught the single biggest structural gap in the system, plus several money- and security-integrity issues.

P0 BUG-006 — Delivery Coordinator had no web portal at all

Login redirected the role into a route that returned a real Next.js 404 — the entire `(delivery)` route group had never been built, despite the backend API being fully functional. Built `apps/web/app/(delivery)/` (dashboard + deliveries list with dispatch/in-transit/POD actions) and fixed the login redirect.

P1 BUG-001 — RMA credit notes never updated the invoice

A resolved RMA correctly inserted an `invoice_adjustments` row, but never recalculated the invoice's own `balance_due` / `status` — a fully-paid invoice stayed marked `PAID` forever even after a credit was issued against it.

P1 BUG-002 — Admin logout panel disconnected from the real logout

Real logout is computed live from `login_history` (5 failures / 15 min); the admin panel read/wrote unrelated columns the login flow never touched. It was permanently blind, and Unlock was a no-op. Reconciled both to the same source of truth.

P1 BUG-003 — Inventory Movements 500'd on every call

Wrong column name (`sm.created_at` vs. the real `occurred_at`) in 4 places — the entire Movements page was unusable.

P1 BUG-004 — Two-person reversal rule accepted a fabricated approver

The >50,000 reversal control only checked that `confirmed_by` was present and not equal to the actor — a completely made-up UUID satisfied it. A single officer could self-authorize any large reversal.

P2 BUG-005 — Dead RFQ convert endpoint, fake 200 success

`PATCH/POST /rfqs/:id/convert` was a scaffold that returned success without creating an order or advancing status. The real UI correctly used `/orders/from-rfq`; the dead route was still live and misleading if hit directly.

P2 BUG-007 — MOQ only enforced at order submission

A below-minimum quantity sat in the cart indefinitely and was only rejected at checkout, discarding the buyer's progress.

P2 BUG-008 — Order approval never recorded who/when

`approved_by` / `approved_at` existed on the schema but were never set — a silent audit-trail gap.

P2 BUG-009 — Delivery assignment accepted any user/role

No validation that `assignedTo` was an actual delivery coordinator; a non-existent id raised a raw 500 with a leaked stack trace.

P3

BUG-010 — Security page swallowed API failures silently

Five data fetches on the admin Security page caught their own errors into an empty array — a failed request looked identical to "no data."

Infrastructure & Environment Fixes

Alongside the QA bug fixes, the environment itself needed repair — a fresh clone couldn't stand the app up cleanly before this session.

- **scripts/migrate.js made idempotent** — now tracks applied migrations in a `schema_migrations` table. Re-running it against an already-migrated DB is a clean no-op instead of failing on the first non-idempotent statement.
- **scripts/seed.js FK-ordering bug fixed** — `rma_requests` was deleted before `invoice_adjustments`, which references it, so re-seeding any existing DB always failed with a foreign-key violation.
- **Missing `apps/api` "build" script added** — the root `build:api` script called a script that didn't exist; `npm run build:api` always failed.
- **3 leftover test/QA products cleaned** from the dev catalogue — marked `DISCONTINUED` rather than hard-deleted, since append-only `stock_movements` / `price_history` triggers correctly blocked outright deletion.
- **README rewritten** to reflect the real state of the system honestly — both QA cycles, known gaps, and setup steps that actually work.

Verified end to end: seeded a completely fresh throwaway database from zero (`migrate` → `seed`), then re-ran `seed` a second time against it with no error — the exact scenario that used to fail.

The Automated Test Suite

Every `*.test.js` file in the codebase was previously an empty placeholder — `describe('X Module', () ⇒ {})`. There was no automated coverage at all; everything above was verified manually.

Built a real **node:test** integration suite — Node's built-in test runner, **zero new dependencies** — that drives the actual Express app against an isolated `project_green_test` database.

`apps/api/src/test/helpers.js`

Starts the real app on an ephemeral port against the isolated test DB, with the env override happening before `config/env.js` is ever required.

`scripts/run-tests.js`

Migrates + seeds the test DB, then runs `node --test` sequentially (`--test-concurrency=1`) — files share seeded fixtures like buyer1's cart, so parallel files would be flaky.

57 tests across 20 modules

auth · cart · orders · payments · rfq · rma · inventory · delivery · invoices · notifications · reports · cms · products · finance · security · buyers · suppliers · users · pricing · tiers

Covers the golden paths (cart→order→approve→invoice, RFQ→quote→convert, partial→final→PAID-at-zero, delivery lifecycle, RMA→credit) **and** explicitly re-asserts every regression from both QA cycles above, so none of the 15 bugs already fixed can silently come back.

Run locally with `npm test`. Never touches the real dev database — the test database name is always derived by suffixing `DATABASE_URL` with `_test`.

8 More Bugs the Suite Caught

Writing real tests against real running code — not just reading it — surfaced genuine bugs no amount of manual spot-checking had caught yet.

P1 Staff user creation was completely broken

`POST/PATCH /users` validated `role_id` as a UUID string — but the real `roles.id` column is a smallint. Every real role id (1–5) failed validation. Admin could not create a single new staff account.

P2 Two wrong SQL column names, both 500ing

`users.repository.findLoginHistory` referenced `attempted_at` (real: `occurred_at`).
`users.repository.findById` joined a non-existent `ta.tier` column (real: join through `tier_id` to `buyer_tiers`).

P2 Supplier creation 500'd whenever `lead_time_days` was omitted

The column has a DB default of 7 — but binding JavaScript `undefined` sends SQL `NULL` , which overrides a column `DEFAULT` and violated the NOT NULL constraint.

P1 The entire CMS content-block module was broken — plus a content leak

Create/update referenced columns (`title` , `block_type` , `image_url` , `link_url` , `start_date` , `end_date` , `meta`) that don't exist on the real `cms_blocks` table — real columns are just `key / type / content (jsonb) / is_published` . Every block create/update 500'd. Separately, **unpublished draft blocks were visible to unauthenticated public visitors** — a real content-leak, not just a broken feature.

P3 Cart's stock check skipped brand-new lines entirely

The check only ever looked at an already-existing cart line — a fresh add of any quantity of an out-of-stock product went through unchecked.

P2 `seed.js` couldn't reseed a DB with real CMS uploads

The clear-tables list never included `cms_media` at all — deleting `users` before `cms_media` hit the same class of FK-order bug already fixed for RMA, just for a table added later.

P3 A JWT timing race in password-change invalidation

JWT `iat` is whole seconds; `password_changed_at` is millisecond-precision. A token issued in the same wall-clock second as a password change could be spuriously flagged "issued before the change" and rejected, even for a fresh login using the new password.

CI/CD on GitHub Actions

A workflow now runs on every push and pull request to `main` and `develop`.

`.github/workflows/ci.yml` — spins up a real Postgres 16 service container, runs `npm ci`, syntax-checks the API, builds the web app, then runs the full 57-test suite. Roughly 11–12 minutes end to end.

P1 First real CI run caught a genuine, previously-invisible bug

The web app is 100% JavaScript — zero `.ts` / `.tsx` files anywhere — with `typescript` not even installed. But it shipped a `tsconfig.json` for its `@/*` import alias instead of the correct `jsconfig.json`. That ambiguous half-TypeScript setup resolved aliases by local-machine accident, but failed outright on a genuinely clean checkout: `Module not found: Can't resolve '@/lib/api'` across dozens of files. Fixed by switching to `jsconfig.json` and removing the stale `tsconfig.json` / `next-env.d.ts`. Verified with a from-scratch `rm -rf .next && next build`.

This is exactly the class of bug CI exists to catch — it never surfaced in any manual "run it locally" check because local state had quietly drifted from what a clean clone actually needs.

Also added `apps/api/.env.example`, documenting every environment variable the API reads (required vs. optional) — there wasn't one before, so a fresh clone had no reference for what to put in `.env`.

Security & Repo Hygiene

P1 .gitignore didn't catch backup env files

`.gitignore` only matched `.env`, `.env.local`, and `*.env` (suffix-only) — a file like `apps/api/.env.qa.bak` (containing real DB/SMTP/JWT values) matched none of those patterns and sat untracked, one `git add -A` away from being committed. Fixed with a `.env.*` pattern, plus an explicit exception for `apps/api/.env.example` (the one `.env`-shaped file actually meant to be tracked). Confirmed it was never present anywhere in git history.



Git Workflow Note

Every fix in this session followed the repository's existing branching convention: committed to `develop`, then merged into `main` with `--no-ff` to preserve a visible merge history, and pushed. No force-pushes, no history rewrites, no direct commits to `main`.

Completion Scorecard

Area	Status
Core commerce flow (all 7 golden paths)	DONE
All 6 role portals (incl. Delivery Coordinator)	DONE
Backend automated test coverage (57 tests)	DONE
CI on GitHub Actions	DONE
Migration / seed reliability (idempotent)	DONE
Secrets / gitignore hygiene	DONE
Frontend automated test coverage	NOT DONE — zero coverage
Product images	NOT DONE — 0 of 523 working
Manual portal-by-portal frontend QA pass	PLANNED — next
Repo hygiene (untracked demo assets/scripts)	NOT RESOLVED
Glassy UI visual polish pass	PLANNED — later

Product images, specifically: the seed script wires up 30 of 523 products with `product_images` rows, but every one points to `res.cloudinary.com/orchids/...` — a Cloudinary bucket that doesn't actually contain those files (confirmed 404 on request). Effectively **zero** products have a working image today, not "30 good, need 70 more."

~ 90 / 100

Overall completion estimate

What's Left To Do

In priority order, based on visible impact for a demo audience versus long-term maintenance value.

1

Source and wire up real product images

Highest visible-impact item — currently 0 of 523 products render an image anywhere in the app. Needs real photography sourced and either a working Cloudinary bucket or the already-built local upload endpoints.

2

Manual portal-by-portal frontend QA pass

Planned as a collaborative, driven-browser session — catches visual/UX issues no automated assertion would catch.

3

Frontend automated test coverage

Currently zero. Lower urgency than the two above for a near-term demo, higher urgency for long-term regression safety across the 5 portals.

4

Repo hygiene cleanup

Decide what to do with the untracked demo videos, screenshots, and one-off build scripts still cluttering the repo root.

5

Glassy UI visual polish pass

Planned for after the above — a polished theme over broken images or untested flows would be the wrong order of operations.

6

Deeper edge-case testing

Reports/BI, notification retry, and credit monitor are currently smoke-tested rather than exhaustively covered — lowest priority of this list.



End of snapshot — ORCHIDS · Project Green, 03 July 2026